

Di-Tunnel — план поддержки Android

Дата: 9 сентября 2026 года. Актуализировано 5 октября 2026 года.

Статус: основные этапы A0–A6 реализованы; подготовлен Android 0.5.13 для ручной проверки как локальный ARM64 APK. Поверх исходного MVP добавлены VMess AEAD, AmneziaWG, плитка, виджеты и четыре языка. На Android 16 подтверждены AWG 2.0 и исправление IPv6; результаты — в [матрице проверки](#). Историческая проверка HarmonyOS 2 относится к 0.4.24 и не подтверждает новый AWG backend. Windows остаётся регрессионной платформой общего кода.

План ниже сохраняет архитектурные цели исходных этапов. В 0.5.10 реализованы сервисное восстановление и запуск системного Always-on (SUPPORTS_ALWAYS_ON=true, Sticky, зашифрованный последний запрос). Lockdown включается пользователем в ОС. Reboot/autoconnect и полный системный прогон восстановления ещё требуют проверки на устройстве. Оставшиеся ограничения AWG перечислены в [проверке реализации](#).

1. Цель и границы

Цель — выпустить rootless VPN-клиент для Android с максимально возможным паритетом с Windows: импорт профилей и подписок, подключение через Xray и AmneziaWG, переключение серверов, Lowest, доменные/IP-правила, маршрутизация по приложениям, восстановление после смены сети, безопасное хранилище и обезличенная диагностика.

Первая поддерживаемая версия — Android 10 (API 29). Сборка компилируется и публикуется с target API 36. Релизный ABI — arm64-v8a; x86_64 используется для эмулятора. Совместимость API 29 подтверждена установкой на HarmonyOS 2. ARM32 можно добавить отдельным решением после проверки спроса, размера и производительности.

На первом этапе не входят:

- root, iptables, TPROXY и скрывание факта использования VPN;
- собственная реализация VPN-протоколов, TLS, QUIC или TCP/IP;
- второй сетевой стек на базе tun2socks, пока встроенный TUN Xray работает корректно;
- обещание полного kill switch силами приложения: гарантированный fail-closed на Android обеспечивает системный режим Always-on VPN с «Блокировать соединения без VPN»;
- автоматическая установка обновлений вне правил выбранного канала распространения.

2. Выбранный стек и закреплённые версии

Компонент	Версия/настройка	Решение
.NET SDK	10.0.302	Сохранить текущую версию из global.json; Android workload ставить из того же workload set
Target framework	net10.0-android36.0	Соответствует .NET 10 и target API 36
minSdk	API 29, Android 10	Покрывает Android-совместимый слой HarmonyOS 2; host и platform-проект используют одну нижнюю границу
targetSdk / compile SDK	API 36, Android 16	Текущая настройка проекта; APK распространяется вручную, публикация в Google Play не выполнена
Avalonia	12.1.2	Оставить единую версию UI-пакетов Windows и Android
Avalonia.Android	12.1.2	Android host и AvaloniaMainActivity

Компонент	Версия/настройка	Решение
CommunityToolkit.Mvvm	8.4.2	Сохранить текущую общую ViewModel-библиотеку
Svg.Controls.Skia.Avalonia	12.0.0.17	Временно сохранить; проверить Android-рендеринг и расход батареи, для проблемных фонов подготовить растровую замену
Xray-core / libXray	Android v26.7.28, Windows v26.3.27	Версии закреплены отдельно; Android получает libXray.aar, Windows — Xray-core
libXray API	JSON Invoke contract из v26.7.28	Обернуть собственным C#-адаптером, не пропускать типы нестабильного upstream API в Core/UI
Go toolchain для воспроизводимой AAR	1.26.3	Зафиксировать вместе с golang.org/x/mobile из release source; не использовать плавающий latest
Тесты	xUnit 2.9.3, runner 3.1.4, Test SDK 17.14.1	Сохранить текущие версии до отдельного обновления всего test stack

Манифест `eng/android-runtime.json` фиксирует `release tag`, `commit`, `URL/имя AAR`, `SHA-256`, `ABI` и версию `Go`. Бинарник нельзя брать по плавающему `URL`. В `CI` нужно либо воспроизводимо собирать `AAR` из закреплённого `source tag`, либо проверять хеш официального `release asset`.

`Xray-core Windows` остаётся на `v26.3.27`; `Android` уже использует `libXray v26.7.28`. Обновление `Windows` требует отдельной регрессии `HY2`, `VLESS TCP/WS/HTTPUpgrade`, `Trojan`, `Shadowsocks`, `TUN`, `DNS`, `IPv4/IPv6`, `split tunneling` и `WFP kill switch`. Различие версий фиксируется в `ADR` и тестовой матрице.

3. Целевая структура решения

`DiTunnel.Core`

Профили, политики, состояния, сценарии, нейтральные интерфейсы

`DiTunnel.Infrastructure.Xray`

Общий разбор ссылок, `outbound/routing JSON`, `GeoIP`, санитизация логов

`DiTunnel.Infrastructure.Xray.Desktop`

`Process/XrayProcessManager` и `desktop SOCKS probe`

`DiTunnel.App`

Общие `Avalonia Views`, `ViewModels`, локализация и адаптивная навигация

`DiTunnel.Platform.Windows`

`DPAPI`, `WFP`, `Windows host`, маршруты, `DNS`, `desktop lifecycle`

`DiTunnel.Android`

`Android Application/Activity`, `manifest`, ресурсы, `DI composition root`

`DiTunnel.Platform.Android`

`VpnService`, `libXray binding`, `TUN FD`, `foreground notification`, `Android Keystore`, `ConnectivityManager`, `app routing` и `Android IPC`

Зависимости направляются только внутрь: `platform/host` проекты зависят от `App/Core/Xray`, но общий код не ссылается на `Android` или `Win32`. Типы `Java/gomobile` не выходят из `DiTunnel.Platform.Android`.

`DiTunnel.Infrastructure.Xray` нужно разделить потому, что сейчас в одном `net10.0`-проекте соседствуют переносимая генерация `JSON` и `System.Diagnostics.Process`. `Android` не должен тащить неиспользуемый `desktop runtime` и связанные проблемы `trimming/AOT`.

4. Что переиспользуем, что рефакторим, что пишем отдельно

Область	Решение
ImportedProfile, parser, Subscription-Userinfo	Переиспользовать без платформенных веток
Загрузка HTTPS-подписок	Переиспользовать через внедряемый HttpClient; добавить тесты Android handler и network handover
Outbound Xray для HY2/VLESS/Trojan/SS	Переиспользовать один генератор и golden tests для обеих платформ
TUN inbound	Платформенные параметры: Windows создаёт Wintun, Android получает FD от VpnService.Builder
Состояния, отмена, переключение, reconnect/backoff	Перенести orchestration в Core/Application и переиспользовать; OS-события поставляют адаптеры
ViewModels и основной экран	Переиспользовать после удаления строк и условий про Windows и внедрения capabilities
Desktop окна, tray, close action, window placement	Оставить Windows-only
Android Activity, VPN consent, service/notification, network callbacks	Реализовать отдельно
Профили и настройки	Общая сериализация/миграции; DPAPI на Windows, Android Keystore на Android
Доменные/IP-правила	Общая модель и Xray routing; фактический сетевой bypass тестируется отдельно на каждой ОС
Правила по приложениям	Общая нейтральная модель; Windows — процессы/позднее WFP, Android — package name через VpnService.Builder
Kill switch	Windows WFP остаётся отдельным; Android показывает состояние системных Always-on/lockdown и не выдаёт мягкую блокировку за гарантированную
Probe/Lowest	Общий алгоритм выбора; Windows использует отдельный процесс, Android — libXray probe/измерение через защищённые сокеты
GeoIP/флаги	Переиспользовать базу и UI-ресурсы; путь к asset получает platform storage provider
Диагностика	Общий формат/редакция секретов; Windows ZIP/file dialog и Android Storage Access Framework/share sheet — отдельно
Проверка релиза/открытие ссылки	Общий HTTP-сценарий; platform launcher отдельно; в Play-сборке учитывать политику магазина

5. Обязательный общий рефакторинг до Android VPN

1. Удалить platform state из статических синглтонов UI. UserSettings.Current, путь через LocalApplicationData и ProfileStorage с P/Invoke DPAPI заменить зависимостями, передаваемыми из composition root.
2. Ввести ISettingsStore, асинхронный IProfileStore, ISecretProtector, IAppDataPaths, IPlatformCapabilities, IExternalLauncher и IDiagnosticsExporter. Асинхронная загрузка выполняется в InitializeAsync, а не в конструкторе ViewModel.
3. Разделить ConnectionPolicy.StartWithWindows на платформенно-нейтральные намерения. UI показывает автозапуск/Always-on только если capability поддерживается ОС.

4. Разделить `SplitTunnelPolicy.Domains` и выбор приложений. В общей модели приложение имеет стабильный `platform ID`; Android хранит `package name`, а не имя процесса.
5. Разложить `XrayProfileConfiguration.Build(bool tun, ...)` на общие `outbound/routing` и платформенный `inbound`. Android-конфигурация не должна получать `Windows tunnel name` или `autoOutboundsInterface` без подтверждённой необходимости.
6. Перенести `XrayProcessManager` и `desktop probe` из общего Xray-проекта в `desktop runtime`.
7. Убрать из `ViewModel` строки «Windows TUN», «нужны права администратора» и «маршруты Windows». Вместо них использовать `display properties/capabilities` платформы.
8. Заменить создаваемые кодом дочерние `Window` в `Dialogs` на общие страницы/`overlay/dialog content`. На Android навигация должна работать внутри `Activity`, с `back button` и восстановлением состояния после `recreation`.
9. Обновить инициализацию Avalonia 12: Android использует `AvaloniaAndroidApplication<App>`, `AvaloniaMainActivity` и `IActivityApplicationLifetime.MainViewFactory`.
10. На Android отключать или упрощать анимацию двадцати SVG-слоёв по умолчанию при `battery saver/reduced motion`; она не должна конкурировать с VPN за CPU.

Этот этап считается завершённым только если все текущие `unit tests` проходят и `Windows VM` повторяет основной `connect/disconnect`, `DNS/IPv6`, `WFP` и `recovery smoke test`.

6. Android VPN runtime

Запуск

1. `Activity` вызывает `VpnService.Prepare`. Если Android возвращает `intent`, приложение показывает системный `consent` через `Activity Result API`; отказ остаётся обычным состоянием, а не ошибкой ядра.
2. После согласия `Activity` запускает `foreground DiTunnelVpnService` явным `intent`.
3. Сервис немедленно создаёт `notification channel` и постоянное уведомление со статусом и действием «Отключить».
4. Сервис определяет активную `underlying network`, резолвит имя сервера до захвата `default routes` и сохраняет IP отдельно от `SNIP/host`.
5. `VpnService.Builder` задаёт `session`, `MTU`, `IPv4/IPv6-адреса`, `DNS` и маршруты `0.0.0.0/0` и `::/0`, затем возвращает `TUN FD`.
6. `FD` передаётся встроенному `TUN Xray` через поддерживаемый `libXray runtime/environment contract`. Все исходящие `TCP/UDP` сокет `Xray` и `Go DNS resolver` проходят через `callback VpnService.Protect(fd)`, иначе возникнет `routing loop`.
7. Статус `Connected` публикуется только после старта `Xray` и успешной проверки трафика через туннель. Один открытый `FD` или флаг `GetXrayState` недостаточен.

Остановка и отказ

- Единственный владелец `TUN FD` и `libXray instance` — `DiTunnelVpnService`.
- Порядок штатной остановки: запрет новых команд → остановка `Xray` → `reset process DNS/controller callbacks` → закрытие `FD` → снятие `foreground notification` → `stopSelf`.
- `OnRevoke`, уничтожение сервиса, отмена подключения и ошибка конфигурации проходят тот же идиоматичный `cleanup`.

- При живом TUN и отказавшем Xray сервис может оставить интерфейс в fail-closed состоянии на ограниченное время reconnect. При завершении процесса Android закроет FD и восстановит обычную сеть, поэтому это не называется полным kill switch.
- Смена Wi-Fi/mobile обрабатывается через `ConnectivityManager.NetworkCallback`: обновить underlying network, DNS и защищённые sockets или выполнить контролируемый reconnect с текущим Core backoff.
- Production-вариант сервиса запускается в отдельном приватном процессе :vpn. UI передаёт команды через версионированный ограниченный Binder/Messenger contract; никаких путей к бинарникам или произвольных команд. Решение закрепляется после измерения памяти на реальном устройстве.

Manifest и безопасность

- service защищён `BIND_VPN_SERVICE`, `exported=false`, intent filter — `android.net.VpnService`;
- foreground service объявлен с подходящим для VPN типом и требуемыми разрешениями target API 36;
- `SUPPORTS_ALWAYS_ON=true` включён в локальной тестовой 0.5.10 для прогона reboot/restore/lockdown; публичный выпуск требует прохождения этой матрицы;
- профили, ключ Android Keystore, runtime configs и сетевые логи исключаются из cloud/device-transfer backup;
- временный Xray JSON хранится только во внутреннем каталоге, не попадает в logcat/exception text и удаляется после сессии; по возможности конфигурация передаётся в память;
- все service intents явные, входные размеры ограничены, package IDs и профили валидируются повторно на стороне сервиса.

7. Особенности функций Android

Kill switch

Начиная с 0.5.10 реализованы Sticky-восстановление и запуск системного Always-on из зашифрованного запроса. Возврат после потери сети подтверждён на телефоне в 0.5.12; текущая версия — 0.5.13. Без системного lockdown после закрытия TUN возможен обычный прямой доступ. Гарантии reboot/restore/lockdown требуют проверки на устройстве; следующие требования задают критерии приёмки.

Android не даёт обычному VPN-приложению программно включить системный lockdown. Экран настроек должен:

- различать «защита при reconnect внутри сервиса» и гарантированное «Блокировать соединения без VPN»;
- показывать фактические `isAlwaysOn/isLockdownEnabled`, если API доступен;
- открывать системные настройки VPN с инструкцией, но не заявлять успех без подтверждения;
- предупреждать, что excluded applications при lockdown могут остаться без сети.

Split tunneling

- Домены/IP остаются правилами Xray.
- «Обход выбранных приложений» реализуется `addDisallowedApplication(packageName)`.

- «Только выбранные приложения через VPN» реализуется `addAllowedApplication(packageName)`.
- В одном Builder нельзя смешивать `allowed` и `disallowed` списки; это проверяет общая `policy validation`.
- Android UI показывает установленные приложения с именем/иконкой, но хранит `package name`. Удалённые `packages` помечаются и не приводят к падению подключения.
- Изменение `app list` требует пересоздания VPN interface; UI показывает короткое переподключение.

Auto-connect и background

В тестовой 0.5.10 системный Always-on запускает сервис с последним зашифрованным запросом; запуск после reboot и восстановление без UI требуют проверки на устройстве. Обычный `boot receiver` не используется. Ручное подключение запускается из видимой Activity или действия уведомления с учётом ограничений `background foreground-service start`.

Probe и Lowest

Сначала реализовать `probe` при отключённом VPN через `libXray`. Затем проверить `batch probe` во время активной сессии без второго конфликтующего `Xray instance`. Если `upstream` не гарантирует это, во время подключения измерять только активный сервер, а полный `Lowest` запускать перед переключением или через контролируемую паузу. UI всегда указывает метод измерения; `HTTPS duration` не называется `ICMP ping`.

8. Этапы реализации

A0. Зафиксировать решения и снять главный риск

- Создать ADR Android runtime и runtime manifest.
- Собрать минимальный `net10.0-android36.0 host` с Avalonia 12.1.2.
- Подключить `libXray.aar v26.7.28`, получить `version`, выполнить `validation` и `SOCKS/HTTP probe`.
- На отдельном тестовом устройстве создать `VpnService TUN`, передать `FD Xray`, проверить `HY2 TCP, UDP и DNS`.
- Проверить `ProtectFd, IPv6, MTU` и отсутствие `routing loop`.

Gate: реальный `HTTP/HTTPS, UDP и DNS` проходят через `Xray TUN`; `disconnect` полностью восстанавливает сеть. Если `gate` не пройден, не начинать продуктивный UI/service и отдельно решить вопрос версии `libXray` или `fallback bridge`.

A1. Подготовить общий код

- Выполнить рефакторинг раздела 5 маленькими коммитами.
- Добавить `contract/golden tests` для одинакового `outbound/routing JSON Windows и Android`.
- Сохранить `Windows runtime` без изменения поведения.

Gate: полный `unit suite` зелёный; `Windows regression matrix` зелёная; в общих проектах нет `Win32/Android API`.

A2. Android shell и мобильный UI

- Добавить `Android projects, Application, Activity, manifest, adaptive/monochrome icon`.

- Реализовать Activity lifetime, in-app dialogs/navigation, back handling, portrait/landscape и keyboard/clipboard.
- Внедрить platform capabilities; скрыть tray/window/start-with-Windows элементы.
- Проверить small phone, tablet, font scaling 1.0/1.3/2.0, RU/EN, dark/light.

Gate: все существующие сценарии профилей и настроек доступны без VPN runtime; Activity recreation не теряет несохранённый ввод и выбранный профиль.

A3. Хранилище и импорт

- Реализовать AES-GCM encrypted profile file; master key — non-exportable Android Keystore key.
- Добавить schema version, atomic write, corrupt-file fail-safe и migration tests.
- Исключить секреты из Android backup.
- Подключить subscriptions, paste/share intent и диагностический export через системный picker/share sheet.

Gate: данные переживают restart/update, не читаются как plaintext и не затираются при ошибке расшифровки; экспорт не содержит profiles/credentials.

A4. VPN MVP

- Реализовать consent, foreground service, notification, Binder contract и AndroidVpnEngine.
- Подключить общий Xray config и протоколы, уже работающие на Windows.
- Реализовать connect/cancel/disconnect/switch/delete-active и статусную модель.
- Поддерживать IPv4+IPv6 и DNS через туннель.

Gate: HY2, VLESS TCP/WS/HTTPUpgrade, Trojan и Shadowsocks проходят protocol matrix на реальном arm64-устройстве; VMess AEAD поддерживан общим converter; его системная проверка на Android входит в protocol matrix. Произвольный JSON доступен только для хранения.

A5. Lifecycle и восстановление

- Обработать screen off, Doze, Activity kill, service restart, VPN revoke, Wi-Fi/mobile handover, airplane mode и reboot/Always-on.
- Реализовать bounded reconnect и идиempотентный cleanup.
- Добавить фактические состояния Always-on/lockdown.

Gate: не остаётся zombie notification/service/FD; после штатной остановки сеть работает; при lockdown нет подтверждённых IPv4/IPv6/DNS leaks во всех сценариях отказа.

A6. Паритет функций

- Перенести Lowest/probes и периодическое измерение активного сервера.
- Реализовать доменные/IP-правила и Android per-app routing.
- Добавить mobile diagnostics и понятные ошибки для consent/revoke/background restrictions.
- Оптимизировать CPU, память, battery и размер AAB; проверить SVG/background animation.

Gate: матрица паритета ниже закрыта либо каждое оставшееся ограничение явно показано в UI/README.

A7. Релиз

- Сначала подписанный internal APK, затем AAB.
- Проверить reproducible inputs, signing secrets вне репозитория, SBOM/third-party notices и SHA-256.
- Добавить CI build/unit tests, emulator smoke без реальных подписок и ручной real-device checklist.
- Подготовить privacy policy/data safety disclosure для VPN и канала распространения.

Gate: clean install/update/uninstall, Android 10 и 16, ARM64 device и x86_64 emulator; release artifacts не содержат тестовых серверов, ключей или полных сетевых журналов. Результаты фиксируются в android-release-validation.md.

9. Матрица паритета

Функция Windows	Android target	Комментарий
HY2, VLESS TCP/WS/HTTPUpgrade, Trojan, SS	Да, A4	Общий converter, Android libXray runtime
VMess AEAD / произвольный JSON	Реализован / хранение	VMess использует общий converter; подключение произвольного JSON не реализовано
AmneziaWG	Реализован, AWG 2.0 проверен на телефоне	Xray TUN + отдельный AWG netstack; ограничения — в проверке поддержки
Плитка, виджеты, четыре языка	Реализованы	Синхронизация управления подтверждена пользователем
Подписки, обновление, группы, удаление	Да, A2–A3	Общие сценарии и ViewModels
Безопасное переключение активного сервера	Да, A4	Service-owned transition
Lowest и периодическая задержка	Да, A6	С учётом single-instance ограничений libXray
Домены/IP: all/bypass/only selected	Да, A6	Xray routing + отдельная leak validation
Split tunneling по приложениям	Да, A6	На Android реализуем раньше Windows через Builder package lists
Kill switch	Требует проверки	Always-on поддерживается сервисом; fail-closed требует включённого системного lockdown и прогона на устройстве
IPv4/IPv6/DNS	Реализованы	AWG объявляет только поддерживаемые семейства; полный прогон утечек для нового backend ещё требуется
Reconnect/network change	Да, A5	ConnectivityManager и foreground service
Трей/заккрытие окна	Не применимо	Заменяет foreground notification
Автозапуск	Реализован, требует проверки	Системный Always-on восстанавливает последний зашифрованный профиль после reboot
Диагностический ZIP	Да, A3/A6	SAF/share sheet, тот же whitelist логов

Функция Windows	Android target	Комментарий
Проверка GitHub release	Sideload build	Play build следует правилам магазина

10. Проверки

Автоматически на каждом PR:

- Core/App/Xray unit tests на desktop;
- одинаковые golden configs обеих платформ;
- Android compile, trimming/linker warnings as errors для нашего кода;
- parser fuzz/property cases, IPC/input size limits, secret-redaction tests;
- emulator API 29 и API 36 smoke: Activity, storage, service lifecycle без реального VPS.

На изолированном Android-устройстве/эмуляторе перед релизом:

- TCP, UDP, DNS, IPv4, IPv6, SNI/certificate validation;
- DNS/IPv6 leak test и сравнение exit IP;
- Wi-Fi → mobile → Wi-Fi, captive/no-network, airplane mode;
- Xray failure, service process kill, UI process kill, VPN revoke, reboot;
- normal mode, Always-on и lockdown;
- allowed/disallowed applications и сочетания с доменными правилами;
- 30 минут нагрузки, screen off/Doze, температура, battery/CPU/memory;
- крупные загрузки, QUIC, MTU/fragmentation и несколько OEM-устройств, если доступны.

Реальные подписки и серверные секреты используются только локально. В CI и репозитории — синтетические profiles и обезличенные результаты.

11. Порядок первых коммитов

1. Добавлен ADR Android runtime
2. Добавлен каркас Android-приложения
3. Разделён общий и desktop runtime Xray
4. Внедрены платформенные хранилища настроек
5. Адаптирована навигация для Android
6. Реализован Android VpnService
7. Подключён Android runtime Xray
8. Реализовано восстановление Android VPN
9. Добавлена маршрутизация Android-приложений
10. Протестирован Android-релиз

Каждый сетевой коммит должен быть меньше одного этапа, иметь автоматические contract tests и отдельный ручной checklist. Обновление Xray не смешивается с реализацией VpnService.

12. Официальные источники решений

Проверены 9 сентября 2026 года:

- [Avalonia: supported platforms](#)
- [Avalonia: Android setup](#)
- [Avalonia 12: Android initialization changes](#)
- [Android: VpnService](#)
- [Android: VpnService.Builder](#)
- [Android: foreground service types](#)
- [Android: target API requirement](#)
- [Xray: TUN inbound and Android FD](#)
- [XTLS/libXray releases](#)

13. Реализованные расширения Android 0.5.9–0.5.13

- AWG и libXray изолированы в процессах :awg и :vpn; временные проверки — в отдельных процессах без системного TUN.
- AWG UDP защищён и привязан к физической Network до входа в Go; IPC не экспортирован, конфигурация передаётся в памяти.
- Плитка и виджеты работают через общий движок, выбор сервера синхронизируется с интерфейсом; отмена доступна при подключении и переподключении.
- Локальный workflow, SDK/NDK, новый ключ подписи и APK находятся в .tools/android-build-local, вне Git. Прежний ключ утрачен; совместимость обновления сохраняется только внутри новой линии подписанных тестовых APK.
- В 0.5.12 подтверждены возврат после потери сети и открытие приложения долгим нажатием плитки. В 0.5.13 добавлен сброс галочек текущего режима PT; версия синхронизирована с Windows.
- PDF-копия этого плана генерируется из Markdown; источником актуальных изменений остаётся этот файл.
- Полный список подтверждённых сценариев и оставшихся задач поддерживается в [проверке AWG](#) и [матрице Android](#).